

SM57: CANOPEN

Complément de cours

III. Le Protocole CAN

12) La couche Applicative CANOpen

CANOpen est une couche applicative (**couche 7 du modèle OSI**), dédié originellement pour les Bus de terrain du type CAN fonctionnant en temps réel. Il est utilisé dans de nombreux domaines : automobile, agricole, industriel (ascenseurs, escaliers roulants, motion control) et médical (rayons X, salles d'opérations). Ce bus de terrain est connu pour être une solution de communication économique et efficace.

CANOpen est une reprise **de la couche applicative CAL** développée par Philips Medical Systems ; il reprend les services et protocoles de gestion de bus et de messages de la couche CAL tout en définissant le contenu des messages et en intégrant la notion de système distribué. Un élément maître du réseau coordonne les éléments esclaves. Il respecte la norme ISO 11898.

III. Le Protocole CAN

A sa création, CAN Open était destiné à des systèmes industriels de contrôle de mouvement ou de manipulation tandis qu'aujourd'hui il est utilisé dans de nombreuses applications comme les véhicules, les transports publics, les équipements médicaux ou l'électronique maritime.

Le premier avantage du protocole CAN Open est qu'il supporte des systèmes temps réel car un temps maximal entre l'émission et la réception des trames pour un processus quelconque peut être défini. Le second avantage est la programmation objet.

III. Le Protocole CAN

a) Les Dictionnaires d'objets (OD)

La partie la plus importante d'un élément CANOpen est l'Object Dictionary. Il s'agit en fait d'une table regroupant tous les objets accessibles via le bus. Chaque objet du dictionnaire est adressé en utilisant un index sur **16 bits** et un sous-index sur **8 bits**.

Index (hex)	Object
0000	Reserved
0001-001F	Static Data Types
0020-003F	Complex Data Types
0040-005F	Manufacturer Specific Data Types
0060-007F	Device Profile Specific Static Data Types
0080-009F	Device Profile Specific Complex Data Types
00A0-0FFF	Reserved for futher use
1000-1FFF	Communication Profile Area
2000-5FFF	Manufacturer Specific Profile Area
6000-9FFF	Standardized Device Profile Area
A000-FFFF	Reserved for futher use

III. Le Protocole CAN

Un dictionnaire d'objet définit, pour chacune des entrées/sorties d'un périphérique CANopen (appelé nœud), l'information sur le format de la donnée ainsi que sur le moyen d'y accéder. Chaque entrée du dictionnaire possède un index unique ainsi qu'une liste de sous-index. On accède à un objet grâce au couple [index, sous-index].

Ce dictionnaire d'objets est généralement sous forme de fichier texte au format EDS (Electronic Data Sheet) permettant de connaître l'ensemble des entrées/sorties d'un nœud. Il est possible de créer à partir d'un fichier au format EDS un autre fichier représentant une configuration donnée d'un nœud pour un bus. Ce fichier, très similaire à l' EDS, est alors appelé DCF (Device Configuration File). En faisant une analogie avec la programmation orientée objet (POO) on peut dire que l' EDS est la classe tandis que le DCF représente une instance de cette classe.

III. Le Protocole CAN



III. Le Protocole CAN

Les dictionnaire d'objets (OD) sont divisés en profiles. Les plus utilisés sont les suivants :

- Communication Profile Area
- Manufacturer Specific Profile Area
- Standardised Device Profile Area

Toutes les entrées de ces profiles sont définies par la CIA (**Can In Automation**) excepté pour le Manufacturer Specific Profile Area qui est libre d'utilisation.

III. Le Protocole CAN

b) Les protocoles de communication

CANopen offre différents moyens pour accéder aux données du dictionnaire d'objets :

- Les transferts de données en temps réel sont réalisés à l'aide de PDO (**Process Data Object**)
- La lecture ou l'écriture d'entrées dans le dictionnaire d'objets d'un élément sont réalisés à l'aide de SDO (**Service Data Object**)
- Les applications telles que la Synchronisation, le Time Stamp, l'Emergency ont des protocoles spéciaux
- Le protocole du **Network Management** (NMT) donne des services pour l'Initialisation, le Contrôle d'Erreurs, et l'Etat d'un Nœud

III. Le Protocole CAN

1. COB et COB-ID

Chaque trame CANOpen commence par un COB-ID faisant office d'**identifiant** de cette trame. C'est au niveau de l'arbitrage, la priorité est attribuée selon la valeur du champ d'arbitrage sur **11 bits** pour le CAN 2.0 A. Les 4 premiers bits de celui ci sont le COB (objet de communication) et les 7 suivants sont l'ID (identifiant). Au cours de la phase de configuration, chacun des noeuds reçoit le COB-ID de la (ou les) trame(s) dont il est le récepteur ou l'émetteur.

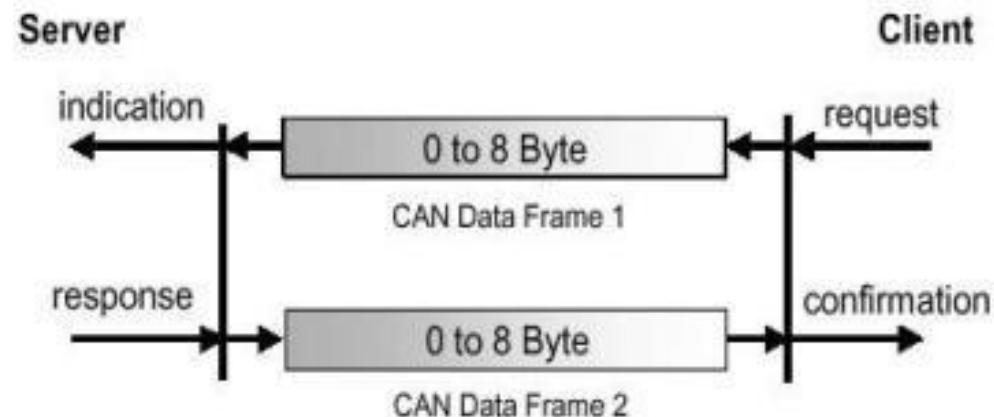
III. Le Protocole CAN

2. Client/Server, Producer/Consumer, Master/Slave

Le modèle Client/Server :

Dans ce modèle, le client transmet un message auquel le serveur répond. Le client obtient ainsi une confirmation.

Ce modèle est utilisé pour transmettre des données dont la longueur est supérieure à 8 octets. En fait la donnée originale est **segmentée et transmise segment par segment**. L'élément recevant les segments peut renvoyer une confirmation après avoir reçu un ou plusieurs segments. Cette communication est donc du type peer-to-peer.

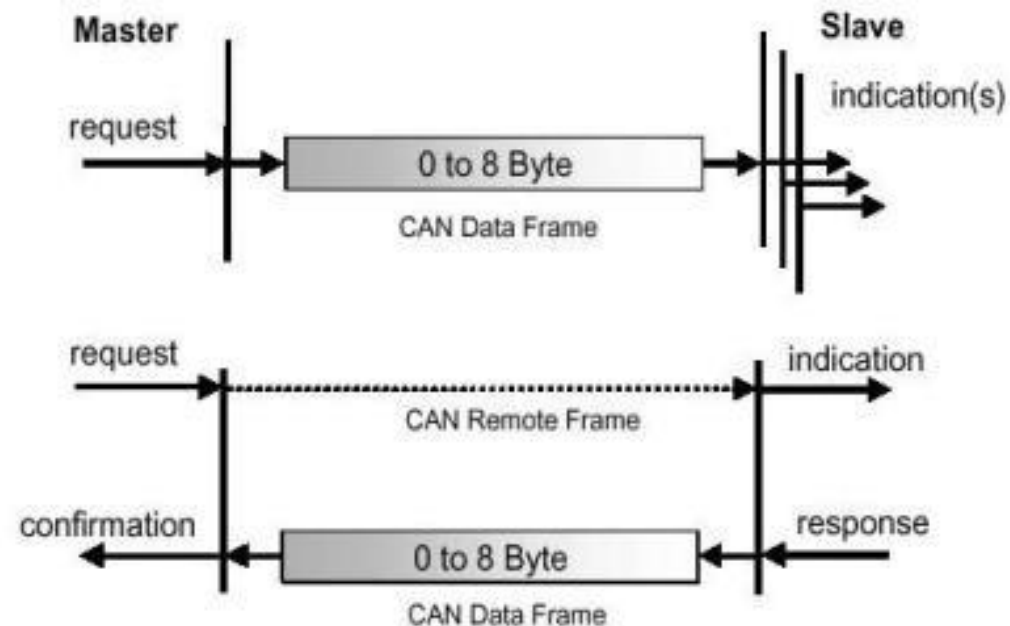


III. Le Protocole CAN

Le modèle Master/Slave :

Dans ce modèle, seul le maître peut commencer une communication et l'esclave est toujours en train d'attendre une requête de communication du maître.

Dans les réseaux CAN, ce modèle peut être implanté en allouant des identifiants appropriés aux différents éléments.

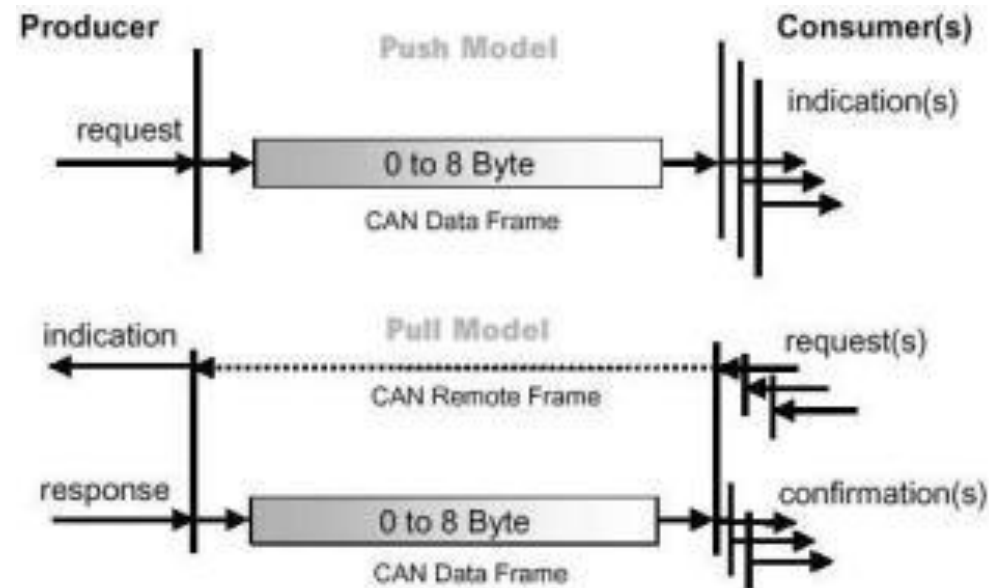


III. Le Protocole CAN

Le modèle Producer/Consumer :

Ce modèle décrit parfaitement les possibilités de broadcast du réseau CAN. En effet, chaque élément du réseau peut écouter les messages qui transitent sur le bus. C'est après avoir reçu le message que l'élément décide si il l'accepte ou non. Le modèle Producer/Consumer permet de :

- Transmettre des messages (modèle Push)
- Demander des messages (modèle Pull)

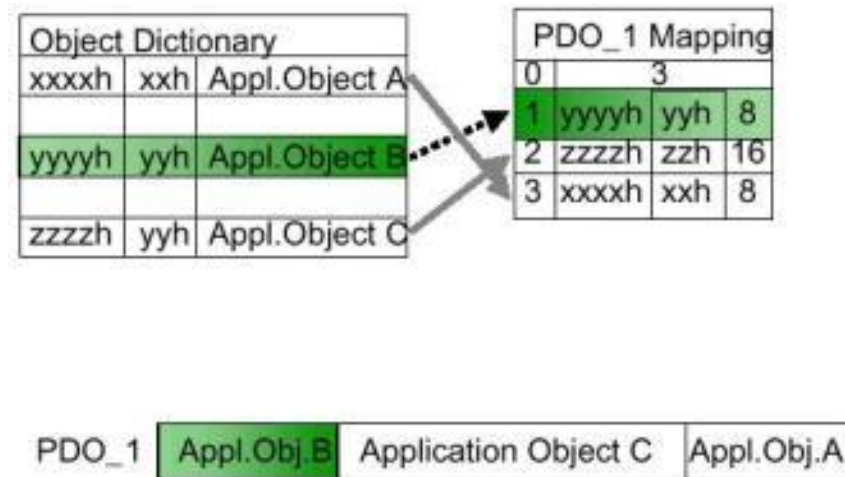


3. PDO (Process Data Object) et PDO Mapping

Il est destiné au transfert de données temps réel, ce protocole permet de mettre dans un même message un ou plusieurs champs du dictionnaire d'objets. On peut donc mettre dans un même message **jusqu'à 64 messages de 1 bit**. Ce type de message réduit donc fortement l'overhead (le nombre de bits ne transportant pas d'information utile). L'autre avantage des PDO est que la lecture se fait au travers d'un message court de type synchrone ou asynchrone qui est reçu par l'ensemble des nœuds présents sur le bus. Ce mécanisme permet de lire une information donnée au même instant sur l'ensemble des capteurs/nœuds du bus.

III. Le Protocole CAN

Les messages PDO sont utilisés pour manipuler des objets du dictionnaire d'objets sans se référencer explicitement à leurs identifiants, les types des données et la liste des objets à transmettre étant définis dans une structure appelée PDO mapping.

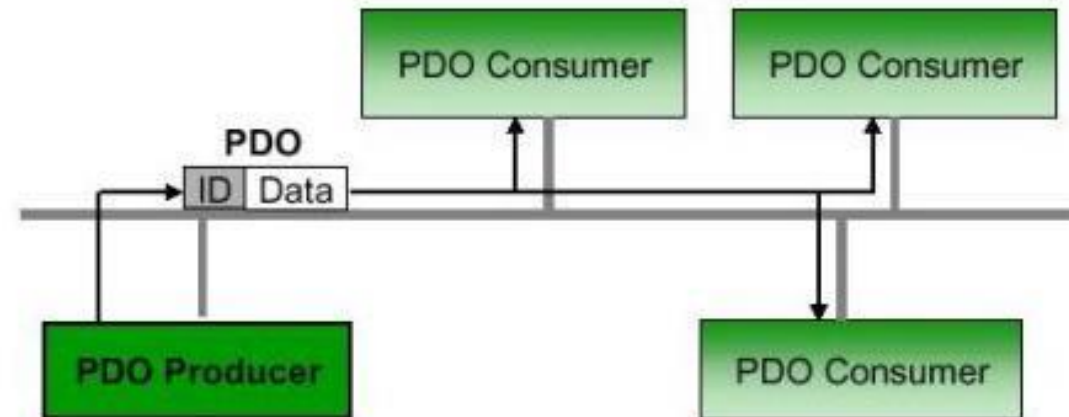


III. Le Protocole CAN

Dans le schéma ci-dessus, le PDO mapping PDO_1 va provoquer la réalisation successive des applications décrites par les objets yyyyh, puis zzzzh et enfin xxxxh

Une communication par PDO ressemble au modèle Producer/Consumer. La donnée est transmise sans demande de confirmation et est émise d'un nœud (Producer) vers un, plusieurs ou tous les nœuds (Consumers).

Le nœud Producer envoie un Transmit-PDO qui correspond, pour le nœud consommateur, à un Receive-PDO.



III. Le Protocole CAN

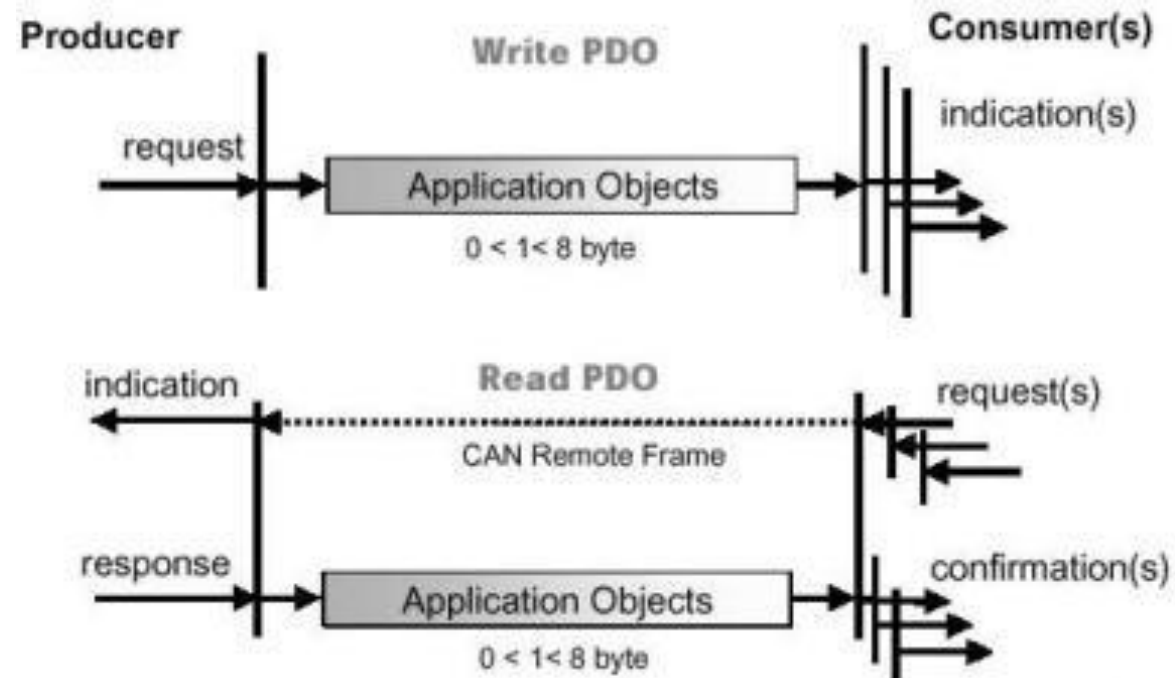
Les messages PDO correspondent aux entrées du dictionnaire d'objets et fourniront à l'application concernée les données nécessaires à son fonctionnement.

Il existe principalement deux types de message PDO:

- Ecrire un PDO : Ce service est contenu dans une seule Data Frame.
- Lire un PDO : Ce service est contenu dans une Remote Frame. La réponse à cette dernière est la Data Frame correspondante.

Dans les deux cas, le champ de données de la trame contient la donnée du process sur 8 octets.

III. Le Protocole CAN



III. Le Protocole CAN

Deux paramètres permettent de décrire un PDO :

- Le **PDO communication parameter** qui décrit les possibilités de communication d'un PDO

Index	Sub-Index	Description	Data Type
1XXXh	0h	Number of entries	Unsigned 8
	1h	COB-ID	Unsigned 32
	2h	Transmission type	Unsigned 8
	3h	Inhibit time	Unsigned 16
	4h	Reserved	Unsigned 8
	5h	Event timer	Unsigned 16

III. Le Protocole CAN

- Le **PDO mapping parameter** qui contient les informations relatives aux contenus des PDO

Index	Sub-Index	Description	Data Type
1XXXh	0h	Number of entries	Unsigned 8
	1h	1 st Object	Unsigned 32
	2h	2 nd Object	Unsigned 32
	...	...	...
	40h	64 th Object	Unsigned 32



III. Le Protocole CAN

Nota 1 : *avant d'effectuer une transmission de type PDO, le numéro des PDO ainsi que la cartographie des objets (mapping) doivent être transmis aux éléments du réseau intéressés pendant la phase de configuration (Device Configuration) c'est à dire pendant l'initialisation. Ceci s'effectue d'ailleurs à travers un message de type SDO.*

Nota 2 : *la relation Producer-Consumer implique l'existence d'un Producer et de zéro ou plus Consumers. Du Producer vers le Consumer c'est le Push model qui est un service dit unconfirmed. Du Consumer vers les Producer c'est le Pull model qui est un service dit confirmed.*

III. Le Protocole CAN

Il existe des PDO synchrones et asynchrones. Les PDO synchrones sont exécutés à chaque transmission du SYNC tandis que les asynchrones peuvent être exécutés à n'importe quel moment du moment que la priorité est respectée.

Il existe 4 types de PDO correspondant à différents niveaux de priorité (PDO1 ayant la priorité la plus élevée) et les objets sont :

Object	COB-ID	Index
Transmettre un PDO1	180h + Node ID(181h à 1FFh)	1800h
Recevoir un PDO1	200h + Node ID(201h à 27Fh)	1400h
Transmettre un PDO2	280h + Node ID(281h à 2FFh)	1801h
Recevoir un PDO2	300h + Node ID(301h à 37Fh)	1401h
Transmettre un PDO3	380h + Node ID(381h à 3FFh)	1802h
Recevoir un PDO3	400h + Node ID(401h à 47Fh)	1402h
Transmettre un PDO4	480h + Node ID(481h à 4FFh)	1803h
Recevoir un PDO4	500h + Node ID(501h à 57Fh)	1403h

III. Le Protocole CAN

Description des différents objets des PDO :

1800h : Transmit PDO Communication Parameter

DATA TYPE	PDO CommPar
OBJECT CODE	RECORD
SUB INDEX 0	Largest sub-index supported
SUB INDEX 1	COBID used by PDO
SUB INDEX 2	Transmission Type
SUB INDEX 3	Inhibit time
SUB INDEX 4	Reserved
SUB INDEX 5	Event timer

III. Le Protocole CAN

1400h : Receive PDO Communication Parameter

DATA TYPE	PDO CommPar
OBJECT CODE	RECORD
SUB INDEX 0	Largest sub-index supported
SUB INDEX 1	COBID used by PDO
SUB INDEX 2	Transmission Type
SUB INDEX 3	Inhibit time
SUB INDEX 4	Compatibility entry
SUB INDEX 5	Event timer

III. Le Protocole CAN***1A00h :Transmit PDO Mapping***

DATA TYPE	PDO Mapping
OBJECT CODE	RECORD
SUB INDEX 0	Number of mapped applications objects in PDO
SUB INDEX 1-40h	PDO Mapping for the nth application object to be mapped

III. Le Protocole CAN***1600h :Receive PDO Mapping***

DATA TYPE	PDO Mapping
OBJECT CODE	RECORD
SUB INDEX 0	Number of mapped applications objects in PDO
SUB INDEX 1-40h	PDO Mapping for the nth application object to be mapped

III. Le Protocole CAN

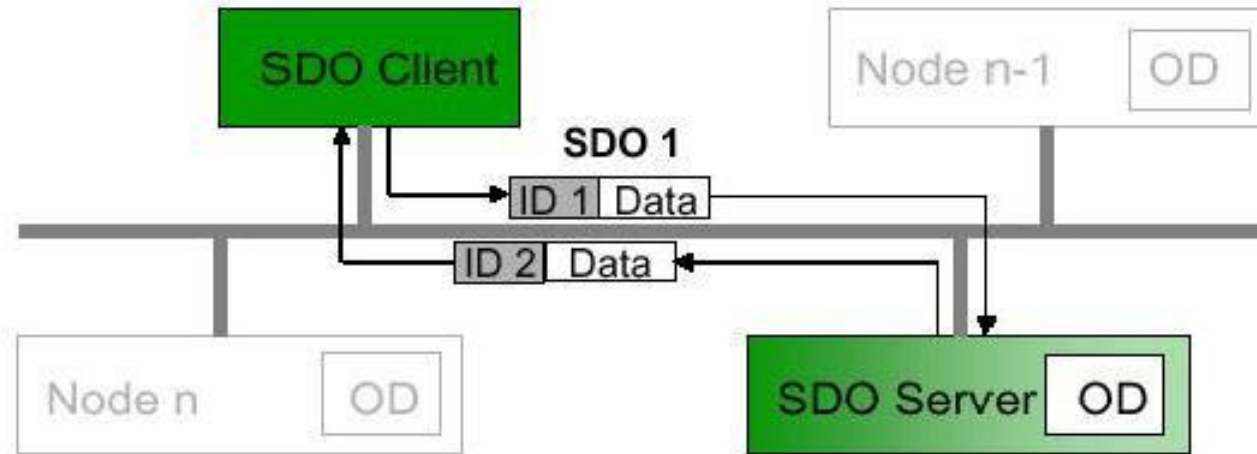
Ces objets sont regroupés dans le tableau suivant :

Receive PDO Communication Parameter (20H)					
1400	RECORD	1 st receive PDO Parameter	PDOCommPar	rw	M/O*
1401	RECORD	2 nd receive PDO Parameter	PDOCommPar	rw	M/O*
xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx
15FF	RECORD	512 th receive PDO Parameter	PDOCommPar	rw	M/O*
Receive PDO Mapping Parameter (21H)					
1600	ARRAY	1 st receive PDO mapping	PDOMapping	rw	M/O*
1601	ARRAY	2 nd receive PDO mapping	PDOMapping	rw	M/O*
xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx
17FF	ARRAY	512 th receive PDO mapping	PDOMapping	rw	M/O*
Transmit PDO Communication Parameter (20H)					
1800	RECORD	1 st transmit PDO Parameter	PDOCommPar	rw	M/O*
1801	RECORD	2 nd transmit PDO Parameter	PDOCommPar	rw	M/O*
xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx
19FF	RECORD	512 th transmit PDO Parameter	PDOCommPar	rw	M/O*
Transmit PDO Mapping Parameter (21H)					
1A00	ARRAY	1 st transmit PDO mapping	PDOMapping	rw	M/O*
1A01	ARRAY	2 nd transmit PDO mapping	PDOMapping	rw	M/O*
xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx	xxxx xxxxx
1BFF	ARRAY	512 th transmit PDO mapping	PDOMapping	rw	M/O*

4. SDO (Service Data Object)

Le système de message SDO suit l'architecture Client-Serveur. Ce service permet l'accès aux paramètres des périphériques (ou nœuds) présents dans le dictionnaire d'objet. Ces entrées peuvent être de grande taille, ils sont donc amenés à transférer plusieurs données appelées **Data Sets**, elles mêmes contenant de nombreux blocs de données. Le client accède à l'objet du dictionnaire d'objets et contrôle quelle Data Set doit être transmis grâce au couple [index,sous-index] et le Serveur répond soit en envoyant les données désirées soit en confirmant le bon déroulement de l'écriture de l'objet. Que ce soit en lecture ou en écriture, le format des données est vérifié et le message contient une confirmation du bon déroulement de la requête.

III. Le Protocole CAN



III. Le Protocole CAN

Il existe deux types de message SDO :

- Le SSDO (Server-SDO) qui est le SDO par défaut
- Le CSDO (Client-SDO)

Dans le cas d'un message SDO on se positionne par rapport au Serveur. Ainsi un **SDO Download** est la transmission de données du Client vers le Serveur et un **SDO Upload** consiste à transmettre des données d'un Serveur vers le Client.

Le SDO permet de transférer des données de n'importe quelle taille et donc le transfert est dit est segmenté.

Les objets sont :

Object	COB-ID	Index
Transmettre un SDO	580h + Node ID (581h à 5FFh)	1200h
Recevoir un SDO	600h + Node ID(601h à 67Fh)	1200h

III. Le Protocole CAN

Description des différents objets des SDO :

1200h : Server SDO Parameter

DATA TYPE	PDO CommPar
OBJECT CODE	RECORD
SUB INDEX 0	Number of entries
SUB INDEX 1	COBID RX
SUB INDEX 2	COBID TX
SUB INDEX 3	Node ID of the SDO client

1280h : Client SDO Parameter

DATA TYPE	PDO CommPar
OBJECT CODE	RECORD
SUB INDEX 0	Number of entries
SUB INDEX 1	COBID RX
SUB INDEX 2	COBID TX
SUB INDEX 3	Node ID of the SDO server

III. Le Protocole CAN

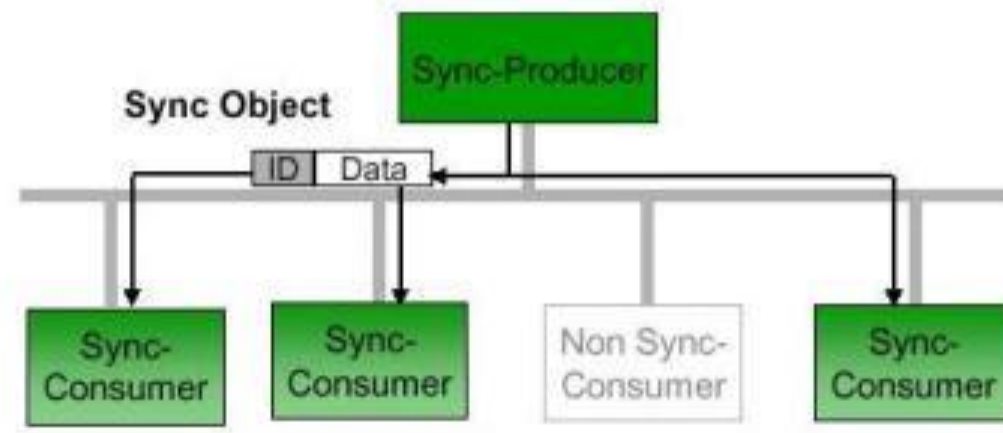
Ces objets sont regroupés dans le tableau suivant :

Server SDO Parameter (22H)					
1200	RECORD	1 st Server SDO parameter	SDOParameter	ro	O
1201	RECORD	2 nd Server SDO parameter	SDOParameter	rw	M/O**
xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx
127F	RECORD	128 th Server SDO parameter	SDOParameter	rw	M/O**
Client SDO Parameter (22H)					
1280	RECORD	1 st Client SDO parameter	SDOParameter	rw	M/O**
1281	RECORD	2 nd Client SDO parameter	SDOParameter	rw	M/O**
xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx
12FF	RECORD	128 th Server SDO parameter	SDOParameter	rw	M/O**
1300		reserved			
xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx	xxxx xxxx
13FF					

5. La synchronisation

Le SYNC est broadcasté périodiquement sur le réseau par le SYNC Producer. C'est l'horloge de base du réseau. Le temps d'une période est défini par le paramètre standard Communication Cycle Period (1006h) qui peut être configuré par un outil de configuration et envoyé à l'application pendant le boot-up. La fréquence de cette horloge peut être temporairement accélérée lorsque certains messages doivent parcourir le réseau avant même que le signal SYNC ne soit transmis. Pour garantir l'accès de SYNC au réseau ce message est doté d'un haut niveau de priorité (1005h).

III. Le Protocole CAN



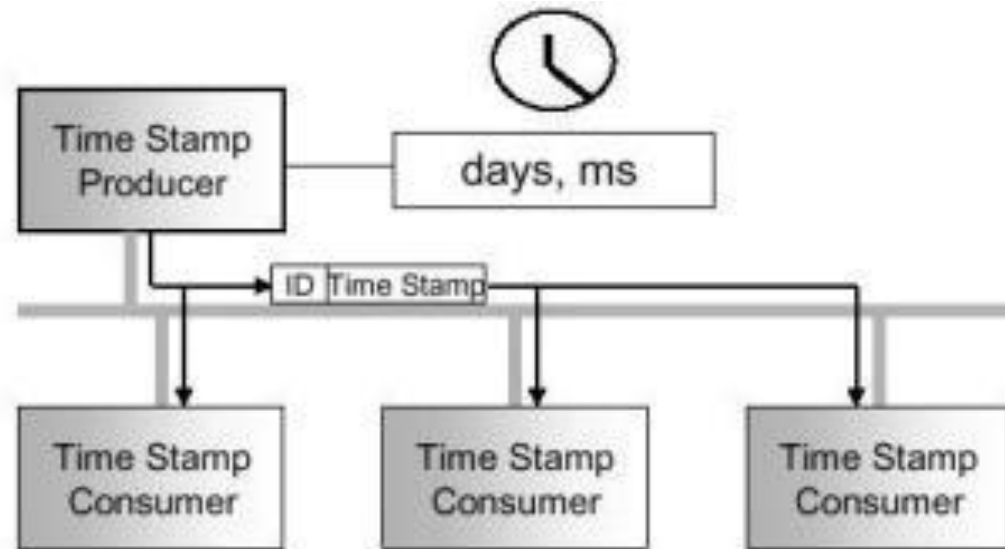
Le premier intérêt de la synchronisation est que lorsque chaque nœud reçoit le signal SYNC, il exécute automatiquement son PDO mapping et envoie donc des données au contrôleur ou exécute une action. Cela permet donc de paramétrer les nœuds pour qu'ils effectuent périodiquement telle ou telle action.

Le second intérêt de la synchronisation est qu'elle permet de synchroniser l'horloge du nœud avec celle du contrôleur : c'est le rôle du Time Stamp (décrit dans le paragraphe suivant) qui contient la valeur de l'horloge maître codée sur 32 bits.

III. Le Protocole CAN

6. Le time stamp

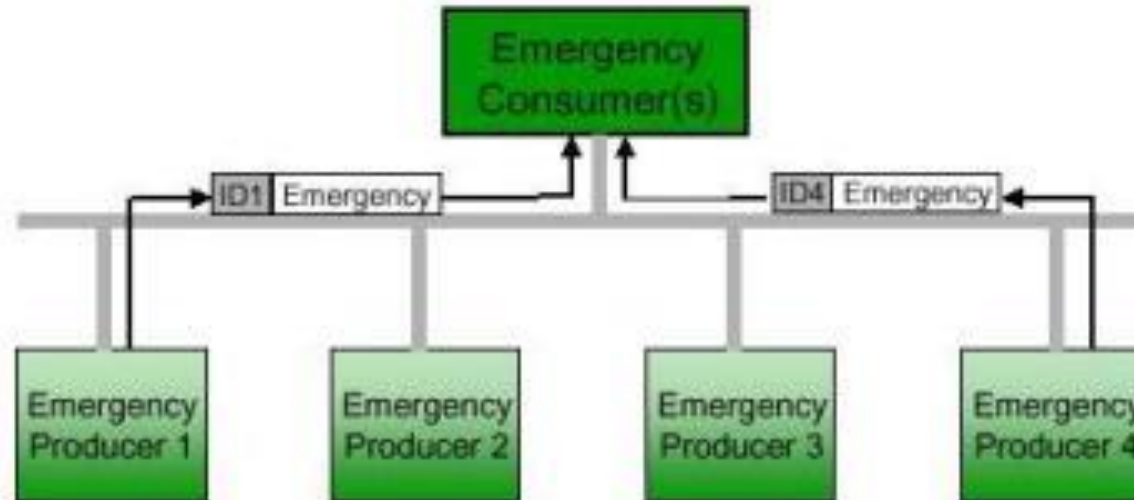
Dans la pratique, l'objet Time Stamp représente en ms le temps écoulé depuis le 1er janvier 1984. C'est une séquence de **48 bits (6 octets)**. Le producteur de Time Stamp reçoit une demande et envoie une Data Frame vers les Consommateurs.



III. Le Protocole CAN

7. L'emergency

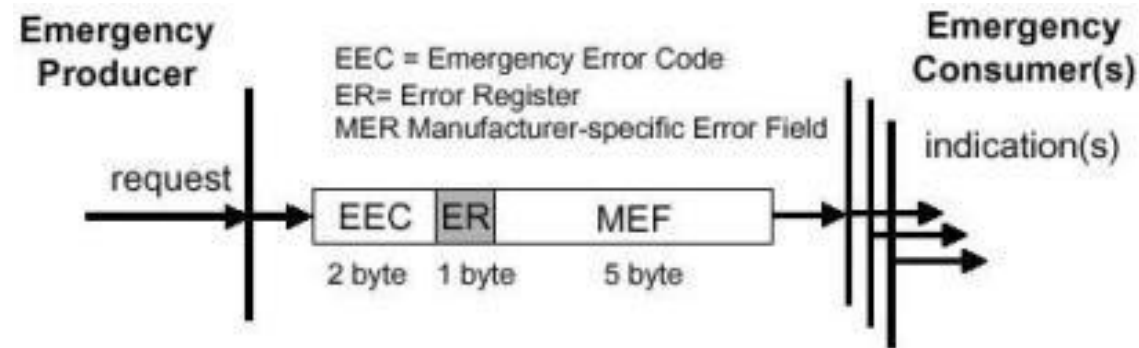
L'emergency permet à un nœud d'envoyer avec une priorité élevée un message contenant le code d'erreur correspondant au problème auquel il est confronté. La table des codes d'erreurs est donnée dans la norme DS-301.



III. Le Protocole CAN

Les 8 bits de data dans la trame sont répartis de la manière suivante :

- Les deux premiers octets contiennent le code d'erreur cité ci dessus
- Le second contient **le registre d'erreur** qui rend compte d'une erreur interne. Il correspond à l'objet 1001h
- Les 5 derniers constituent le **champ d'erreur spécifique au constructeur**



8. Le network management (NMT)

Le NMT est un nœud dit orienté et suit la structure maître esclave. A travers les services du NMT il est possible d'initialiser un nœud, de le démarrer, de faire un reset, de le stopper, de l'activer, de le contrôler.

Tous les nœuds sont considérés comme des esclaves du NMT et un tel esclave est uniquement identifié sur le réseau par son Node-Id, qui est une valeur comprise entre 1 et 127.

Le principe du NMT demande à ce que un élément du réseau remplisse les fonctions de maître NMT. Les services NMT sont :

III. Le Protocole CAN

- **Module Control Service (MCS)**

C'est grâce au MCS que le maître NMT contrôle les différents états des esclaves NMT.

Le rôle de MCS peut être appliqué à un nœud particulier ou à tous les nœuds simultanément.

On remarquera que le maître NMT contrôle son propre état via un service local et que le MCS peut être activé par l'application locale (sauf dans le cas du Start Remote Mode)

- **Error Control Service (ECS)**

A travers ce service le NMT détecte les erreurs du réseau CAN.

Les ECS interviennent principalement lors de défaut dans la périodicité de transmission de message d'un élément du réseau. Il existe deux cas où l'ECS va jouer son rôle :

- ❑ Le maître NMT établit une surveillance des nœuds du réseau (**node guarding protocol**) : si un esclave NMT n'a pas répondu dans un laps de temps imparti (**node life time**) ou encore si ce même esclave change sa politique de communication, le maître en informe sa propre application.

III. Le Protocole CAN

Nota : Principe de la surveillance de l'esclave par le maître: si le Life Time est supporté, l'esclave utilise le Life Time précisé dans son dictionnaire d'objets pour déterminer le Life Time du nœud sur lequel il est connecté, et dans le cas d'une erreur relative à ce temps il en informe sa propre application. Cette surveillance est généralement activée lors de la phase du boot-up.

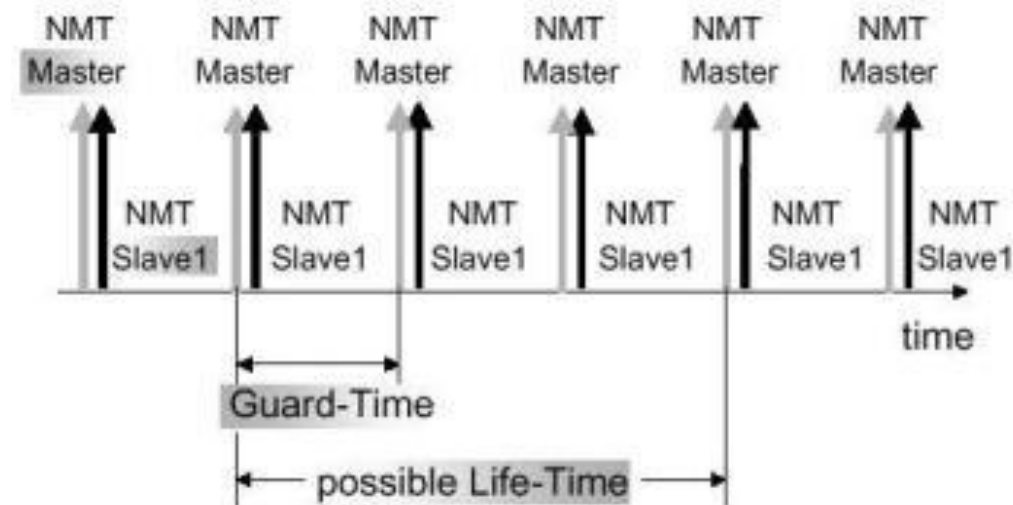
- ❑ Le mécanisme de heartbeat consiste pour un élément de transmettre de façon cyclique un message de présence généré par un Heartbeat Producer. Un ou plusieurs éléments connectés au réseau reçoivent ce message et si sa périodicité change par rapport à celle du Heartbeat Producer alors l'application locale en est informée.

Nota : l'implémentation du Heartbeat ou du Node Guarding est obligatoire.

III. Le Protocole CAN

9. Le node guarding

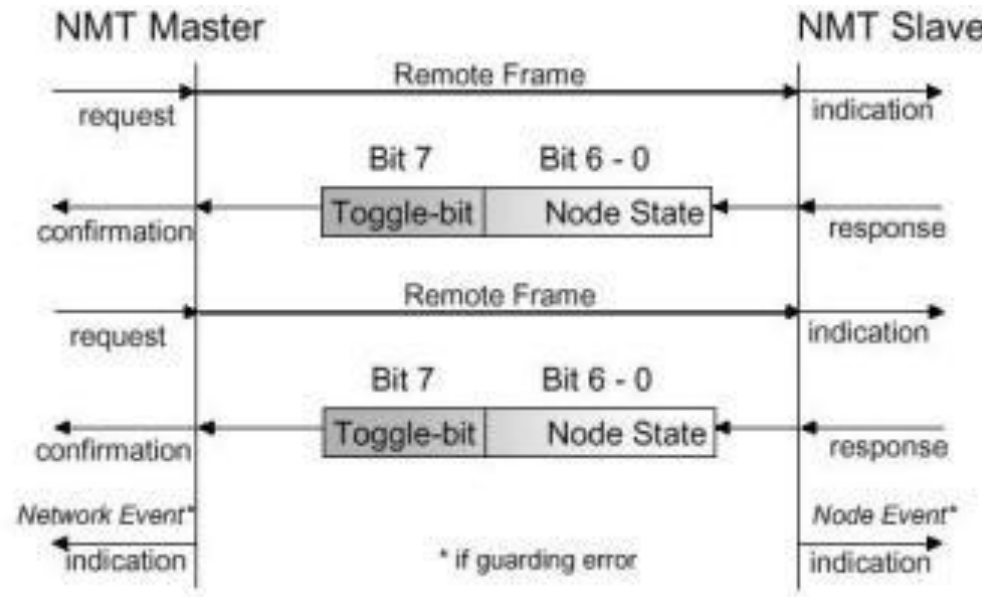
Le Node Guarding est la surveillance des nœuds du réseau. Le maître du NMT (Network Management) envoie une RTR (**Remote Transmission Request**) à intervalles réguliers (**cette période est appelée le Guard Time**) et le nœud concerné doit répondre dans un laps de temps imparti (**c'est le Node Life Time égal au Guard Time multiplié par le Life Time Factor**). La valeur du Life Time est 0 par défaut (c'est à dire qu'il n'est pas utilisé).



III. Le Protocole CAN

Le bit de poids fort (7) est appelé **Toggle bit**. Sa valeur doit changer entre deux réponses consécutives. Lors de la première réponse il est à 0, puis 1, puis 0, etc... Si sa valeur est constante sur deux réponses consécutives, le maître du NMT considérera qu'il n'a pas reçu de réponse.

Nota : Il est interdit d'utiliser en même temps le Node Guarding et le Heartbeat. Si le Heartbeat producer time n'est pas égal à 0, alors le mécanisme du Heartbeat est utilisé.



III. Le Protocole CAN

10. Le Heartbeat

Le Heartbeat est généré périodiquement (la période est le Heartbeat Producer Time) par un nœud sans qu'il ait reçu au préalable une RTR (Remote Transmission Request). Il permet au nœud d'indiquer aux autres son état.

La data est contenue dans un seul octet :

- 0 : Boot-up
- 4 : Stopped
- 5 : Opérationnel
- 127 : Pre-Operational

Si un nœud n'a pas reçu de message de Heartbeat pendant un certain temps (appelé Heartbeat Consumer Time), un événement Heartbeat sera généré.

III. Le Protocole CAN

